Introduction to Scientific Programming

Lecture 2:

Programming Fundamentals

Renato Duarte

@Renatocfduarte rcfduarte.github.io

@rcfduarte renato.duarte@cnc.uc.pt

CENTER FOR NEUROSCIENCE AND CELL BIOLOGY
UNIVERSITY OF COIMBRA
PORTUGAL

1 2 9 0 UNIVERSIDADE D COIMBRA

Organization

Before we get started



- This is the most important lecture of the whole course:
  - make sure you follow
  - don't be shy, interrupt when something is not clear
  - try out the code snippets so you understand how things work

- We start with Part 0: Setting up and running Python
- Goal: everyone can run Python on their own computer before we write any real code
- Stuck? Plan B is Google Colab, and we fix your setup after class

Day 2: Programming Fundamentals

Lecture:

1. Variables, data structures and basic I/O
2. Operators, expressions and control structures
3. Functions and modular programming
4. ~~Classes and basic object-oriented programming for (neuro)scientists~~

moved to intermediate programming concepts

Practical:

- Detailed handout (consolidate knowledge)
- Tutorial exercises

Part 0: *Setting up*

Lecture:

- Install Python and VS Code
- Use the terminal
- Create a virtual environment (more next week)
- Run code in 4 ways
- Read and understand error messages
- Checkpoint: everyone can run code

GOAL: by the end of part 0, everyone can run Python on their own machine

Part 0: *Setting up*

The big picture



Interpreter

the program that reads and runs Python code

`python`



Editor

where you write code, which is just text

`VS Code`



Terminal

where you type commands to run programs

`Anaconda Prompt / Terminal`

Your code is just text. The interpreter reads it and runs it.

Part 0: *Setting up*

Install Python (miniconda)

Miniconda = Python + **conda**, a tool for separate environments.
Download: **docs.anaconda.com/miniconda**

Windows

1. Run the installer
2. Choose **Just Me**
3. Open **Anaconda Prompt** from the Start menu
4. Use it for **all** course commands

macOS

1. Pick Apple Silicon or Intel
2. Run the .pkg installer
3. Open a **new** Terminal
4. conda not found? `conda init zsh`, reopen

Linux

1. `bash Miniconda3-...sh`
2. Accept the defaults
3. Say yes to initialise conda
4. Open a **new** terminal

Never use plain Command Prompt or the Microsoft Store Python.

Part 0: *Setting up*

Install VS Code

1. Download from **code.visualstudio.com** and install with the defaults
2. Open VS Code, click the **Extensions** icon (four squares, or Ctrl+Shift+X / Cmd+Shift+X)
3. Search for **Python** (publisher: Microsoft) and click **Install**
4. Later (Day 3): the **Jupyter** extension



VS Code is the editor, not Python itself: it needs to find Python (coming up).

Part 0: *Setting up*

Check the installation

Open **Anaconda Prompt** (Windows) or **Terminal** (macOS, Linux) and type:

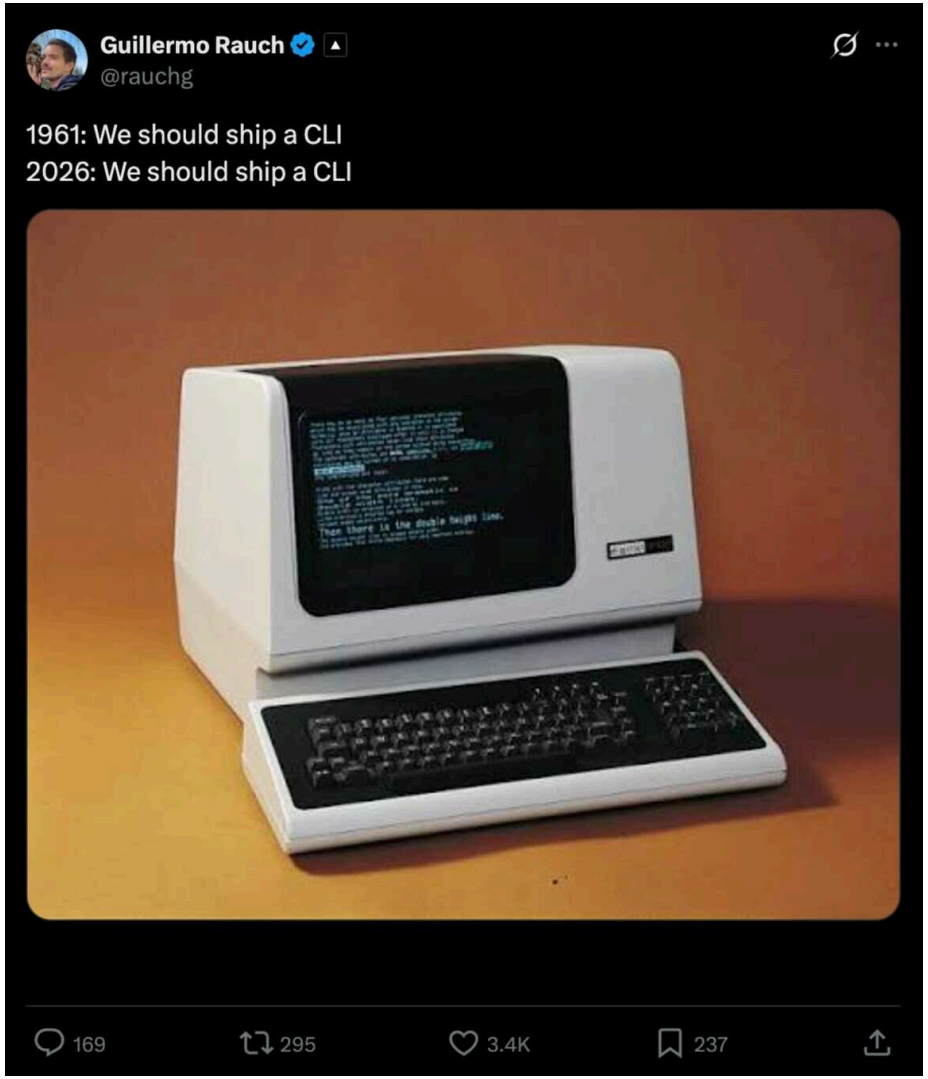
```
$ python --version
Python 3.12.4
$ conda --version
conda 24.5.0
```

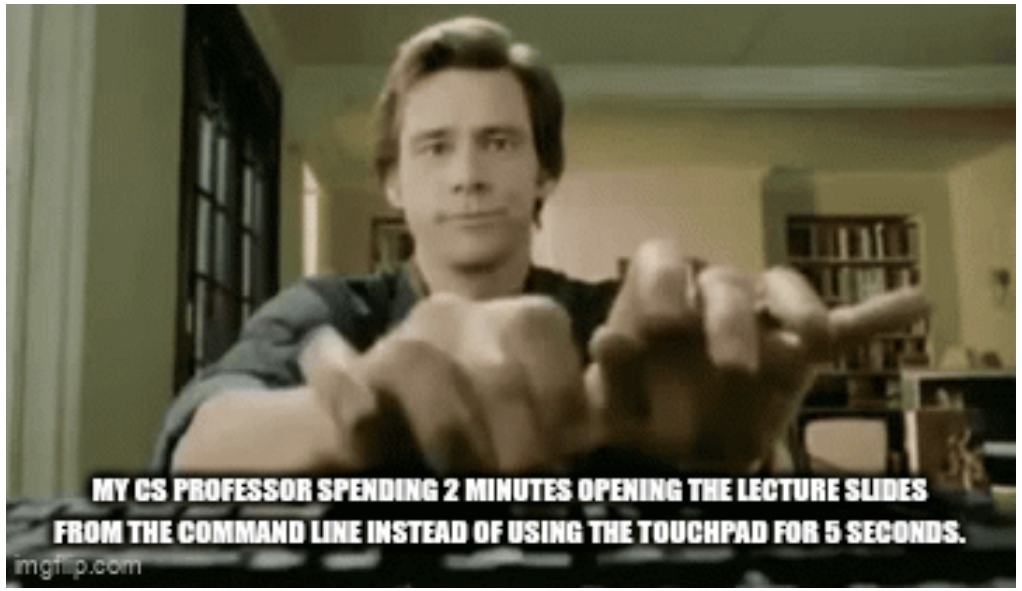
Version numbers will differ. Any Python 3 and any conda version are fine for now.

Something not found?
Windows: you are not in **Anaconda Prompt**.
macOS and Linux: open a **new** terminal, or run `conda init` and open a new one.

Part 0: *Setting up*

The terminal is your friend





Part 0: *Setting up*

The terminal is your friend

What you want	Windows	macOS / Linux
Where am I?	`cd`	`pwd`
List the files here	`dir`	`ls`
Go into a folder	`cd folder_name`	`cd folder_name`
Go up one folder	`cd ..`	`cd ..`
Home folder	`cd %USERPROFILE%`	`cd ~`
Make a folder	`mkdir name`	`mkdir name`
Run Python / a script	`python` / `python script.py`	`python` / `python script.py`
Which Python?	`where python`	`which python`
Stop a running program	`Ctrl+C`	`Ctrl+C`

Tab completes names · **Up arrow** repeats the last command

Part 0: *Setting up*

Where am I?

The terminal always has a **current folder**.

```
$ pwd
/Users/ada/Documents/intscipro
$ ls
README.md day02-fundamentals environment-base.yml ...
$ cd day02-fundamentals/setup
$ python hello.py
```

Scripts and files are found **relative to the current folder**.

Most “file not found” errors mean **you are in the wrong folder**.

Part 0: *Setting up*

Environments

What is an environment?

An isolated workspace with its own Python and its own packages, like a clean bench for each project.

Why?

Projects stay independent and reproducible. Breaking one never breaks another.

```
$ conda activate scientific-programming
(scientific-programming) $ python --version
(scientific-programming) $ conda deactivate
```

The name in brackets tells you which environment is active.

Part 0: *Setting up*

Create your environment

1. Download the course files (GitHub, green **Code** button, **Download ZIP**) and unzip. **2.** `cd` into that folder. **3.** Type:

```
$ conda env create -f environment-base.yml
$ conda activate scientific-programming
(scientific-programming) $ python --version
Python 3.11.x
```

No conda?

`python -m venv .venv`

activate it, then

`pip install -r requirements-base.txt`

The environment is **minimal on purpose** (Python and IPython). More packages arrive when we need them.

Part 0: *Running code*

Four ways to run Python code

1
Interactive prompt
type a line, see the result

2
Script in the terminal
python hello.py

3
VS Code
the run button

4
Notebook
preview, then Day 3

We try all four. You will use 2 and 3 most of the time.

Part 0: *Running code*

1. The interactive prompt

Type `python` (or `ipython`) in the terminal, with the environment active:

```
>>> 2 + 3
5
>>> name = "Ada"
>>> print(f"Hello, {name}")
Hello, Ada
>>> exit()
```

Good for: trying one thing quickly, checking how something behaves

Not saved. Leave with exit() (Ctrl+D on macOS and Linux, Ctrl+Z then Enter on Windows)

Part 0: *Running code*

2. A script from the terminal

A **script** is a text file with Python code, usually ending in `.py`.

```
$ cd day02-fundamentals/setup
$ python hello.py
Hello from Python!
What is your name? Ada
Nice to meet you, Ada. Your first script ran successfully.
```

Error: can't open file ... No such file or directory?
You are in the wrong folder. Check with `pwd` (`cd` on Windows) and `ls` (`dir`), then `cd` to the folder with the script.

Part 0: *Running code*

3. VS Code

1. **File, Open Folder**: choose the course folder
2. Open `hello.py` from the file list
3. Command Palette (Ctrl+Shift+P / Cmd+Shift+P), type **Python: Select Interpreter**, choose the one with **scientific-programming**
4. Click the **play button** at the top right (Run Python File)

Output appears in the **terminal panel** at the bottom (open it with Ctrl+backtick).

Wrong Python or no play button? The **interpreter** is not selected. The active one is shown at the bottom right.

Part 0: *Running code*

4. JuPyTeR Notebooks (preview)

A **notebook** mixes code, results and text in one document. We install and use Jupyter on **Day 3**.

	Interactive prompt	Script	Notebook
Saved as a file	no	yes	yes
Good for	quick tests	programs you rerun	exploring, documenting
Starts fresh each run	n/a	yes	no, the kernel keeps variables

Want to peek now? Google Colab runs notebooks in your browser: **colab.research.google.com**

Part 0: *Running code*

Reading error messages

Errors look scary but are informative. **Read from the bottom.**

```
Traceback (most recent call last):
  File "broken_2.py", line 4, in <module>
    print("Twice as many:", neuron_cont * 2)
NameError: name 'neuron_cont' is not defined
```

1. Last line
the type of error and what went wrong

2. Line above
the file and the line number

3. Look at that line
typo? missing piece? wrong thing?

Part 0: *Running code*

Most common errors

Message	Usually means	Try
SyntaxError	not valid Python (missing colon, bracket, quote)	the line shown and the one before it
NameError	misspelled name, or used before defined	compare spelling; run earlier lines first
ModuleNotFoundError	wrong module name, or not installed here	spelling; is the environment active?
FileNotFoundError	Python cannot find the file from this folder	cd to the right folder or fix the path
IndentationError	inconsistent indentation	4 spaces per level
TypeError	operation does not fit the types, e.g. "5" + 3	print the values and type()
'python' is not recognized / command not found	the terminal cannot find Python	Windows: Anaconda Prompt. macOS, Linux: new terminal, conda init

Part 0: *Checkpoint*

Everyone leaves Part 0 with all five done:

1. `python --version` works in the terminal
2. The environment **scientific-programming** is active (name in the prompt)
3. `python day02-fundamentals/setup/check_python.py` ends with **RESULT: … ready**
4. `hello.py` runs from the terminal **and** from VS Code
5. You fixed the four **broken_*.py** scripts (Exercise 0.3)

Work in pairs. One types, one reads the error. Swap.

Stuck? Raise a hand. Do not wait: error text first, then ask.

Part 0: *Checkpoint*

Plan B

Google Colab

1. Open **colab.research.google.com**
2. Sign in with a Google account
3. New notebook
4. Type `print("Hello")` and press Shift+Enter

What you can do there

- Follow the whole day: paste code into cells
- input() works in a cell
- Fix your computer after class, with us

A broken setup is normal and fixable. It is not a sign you cannot do this.

Part 0: *Done*

You can now

- run Python interactively, as a script, and from VS Code
- move around the terminal and know which folder you are in
- tell which environment is active
- read an error message from the bottom up

Now: Python itself. Variables and data types.

Variables

What are variables?

- Variables are ***symbolic names*** for entities in your program
- Variable names in Python may contain:
  - ASCII characters (i.e. no umlauts, ...)
  - Numbers (but not at the beginning)
  - The underscore character: _
- Examples:

```
a, x, deg_celcius, var1, myVariable, MyVariable
```

- Variable names should be meaningful
- Variable names are *case sensitive*

Variables

Reserved Keywords

- Some words (keywords) are reserved for Python and cannot be used as variable names
- Examples: `for`, `if`, `print`, `or`, `while`, `def`

- Variables can be deleted manually using del although in practice we hardly ever need this (*garbage collection*)
- Example:

```
1 my_var = 23
2 print(my_var)
```

```
23
```

```
1 my_var = 23
2 print(my_var)
3 del my_var
4 print(my_var)
```

```
Error
```

Variables

Assignment

- Variables are defined by *assigning a value* to them
- These statements are called
- Examples:

```
x = 5
y = "Hello world!"
z = x
y = 4
my_pi = 3.1415
sum1 = x + 6
c = 5 + 4i
c = complex(x, y)
```

Not mathematical equality!

Assign the value of x to z
Overwrite y

Two equivalent ways of writing a complex number

Variables

Naming Conventions

- **camelCase**: First word lowercase, subsequent words capitalized
  - Common in Java/JavaScript, but not preferred in Python
- **PascalCase**: All words capitalized, including the first
  - Used in Python for class names
- **snake_case**: All lowercase with underscores
  - Python's preferred convention for variables and functions

```python
# CamelCase
userLoginData = "example"

# PascalCase
class UserProfile:
    pass

# snake_case (preferred)
user_login_data = "example"

def calculate_area():
    pass
```

- Follow PEP 8 guidelines for Python naming
- Consistency improves code readability

`myVariableName, MyClassName, my_variable_name`

Variables

- Each variable has a **type**
- Examples: `int, long, float, complex, str, list, dict`
- The type of a variable depends on its content

```
x = 5
y = 5.0
z = "Hello World"
a = False
```

x is an integer (int)
y is a floating point number (float)
z is a character string (str)
a is a boolean (bool)

- The type of a variable can be queried

```
type(x)
print(type(x))
```

```
<class 'int'>
```

Variables

- Each variable has a **type**
- Examples: `int, long, float, complex, str, list, dict`
- We can **cast** a variable from one type to another

```
x = '5'
x = int(x)
x = float(x)
y = '5.2'
y = int(y)
```

type(x): str
type(x): int
type(x): float
type(y): str
err

- The commands for casting are given by the type names

```
int(), float(), str(), list(), ...
```

Fundamental Data Structures

Strings & indices

- Strings can be specified using single or double quotes
- Examples:

```
str1 = "This is a 'short' string"
str2 = 'This is "also" a string'
```

- Strings are *immutable*: you can't change a string once you've created it
- Strings are a *sequential data type*: you can access individual elements via their index

```
my_string = 'Hello world'
print(my_string[0])
print(my_string[1])
print(my_string[-2])
```

indexing starts from 0

you can count backwards with negative indices

Fundamental Data Structures

Strings & slices

- You can get sub-strings by *slicing*, with the syntax [b:e:s]
- Extracts the elements with indices starting from begin, up to, but not including end, using a given step

```
1 my_string = "Hello world"
2 s1 = my_string[0:5]
3 print(s1)
```

```
>> Hello
```

```
1 s2 = my_string[6:-1]
2 print(s2)
```

```
>> worl
```

```
1 print(s1 + s2)
```

```
>> Helloworl
```

concatenation with +

- If you leave begin blank, the slice starts from 0
- If you leave end blank, the slice ends in the final element
- If you leave step blank, the slice is done in steps of 1

```
1 print(my_string[::2])
```

```
>> Hlowrd
```

Fundamental Data Structures

Lists

- Lists are a form of compound data type that groups values
- Examples:

```
1 lst1 = [1, 2, 3, 4]
2 lst2 = ['red', 6.2, 4, ['a','b'] ]
3 lst3 = []
```

- Like strings, lists can be sliced and concatenated (are *sequential data type*)

```
1 lst1[0]                    >> 1
2 lst1[:3]                   >> [1, 2, 3]
3 lst4 = lst1 + lst1         >> [1, 2, 3, 4, 1, 2, 3, 4]
```

- Unlike strings, lists can be changed (are *mutable*)

```
1 lst1[0] = lst2[0]          >> ['red', 2, 3, 4]
2 lst1[:2] = ['x', 'y']      >> ['x', 'y', 3, 4]
3 lst2[:] = []               >> []
```

Fundamental Data Structures

Tuples

- Tuples are another form of compound data type that groups values
- Examples:

```
1 tpl1 = 4, 17.5, 'hello'
2 tpl2 = ()
3 tpl3 = 'elephant',
```

empty tuple

singleton

- Like strings and lists, tuples can be sliced and concatenated (are *sequential data type*)

```
1 tpl4 = tpl2[:2]
```

```
>> (4, 17.5)
```

- Unlike lists, but like strings, tuples cannot be changed (are *immutable*)

```
1 tpl1[0] = 5
```

```
>> error
```

Fundamental Data Structures

- Dictionaries are associative data types consisting of **key:value pairs**
- Examples:

```
email_dict = {'renato': 'renato.duarte@cnc.uc.pt', 'user1': 'user1@cnc.uc.pt'}
```

or

```
email_dict = dict(renato='renato.duarte@cnc.uc.pt', user1='user1@cnc.uc.pt)
```

```
email_dict['renato']                    >> 'renato.duarte@cnc.uc.pt'
```

```
1 email_dict['user2'] = 'user2@cnc.uc.pt'
2 email_dict.update({'user3': 'user3@cnc.uc.pt', 'user4': 'user4@cnc.uc.pt', 'user5':
  'user5@cnc.uc.pt'})
```

Fundamental Data Structures

- Dictionaries are associative data types consisting of **key:value pairs**
- They are not sequential data types, so they cannot be sliced

```
email_dict[0]                    >> error
```

- Internal methods can be used to obtain lists of elements from the dictionary. Example:

```
email_dict.keys()                >> ['renato', 'user1', 'user2', 'user3', ...]
email_dict.values()              >> ['renato.duarte@cnc.uc.pt', 'user1@cnc.uc.pt', ...]
email_dict.items()               >> [('renato', 'renato.duarte@cnc.uc.pt'), ('user1',
                                 'user1@cnc.uc.pt'), ...]
```

Basic I/O

Reading user input

- Data input can be read into variables

```
var = input()
```

read in one line

- You can add a question to prompt the user for the right input

```
x = input("Your name: ")
y = input("Your age: ")
```

type(x) is str
type(y) is str

- To use the input in calculations, it has to be cast into the right type

```
z = y + 20
y = int(y) + 20
```

error
OK!

Basic I/O

Output to screen

- Variables can be printed on the screen

```
x = 5
print(x)
```

- Multiple things can be printed in one expression

```
print("The value of x is", x)
```

- Special characters can help format the output nicely

`\t`	tabulator
`\n`	new line

Basic I/O

Printing Variables

- **Basic printing:** Just the variable

```
name = "Python"
print(name)
```

- **Comma separation:** Multiple items with spaces

```
print("I like ", name, " Programming")
```

- **String concatenation:** Using + operator

```
print("I like " + name + " programming")
```

Note: With concatenation, you must manually add spaces and convert non-strings with str()

Basic I/O

String Formatting

- **.format() method:** Using placeholders {}

```
name = "John"
age = 25
print("Hello {}, you are {} years old".format(name, age))
```

- **f-strings (recommended):** Modern Python

```
print(f"Hello {name}, you are {age} years old")
```

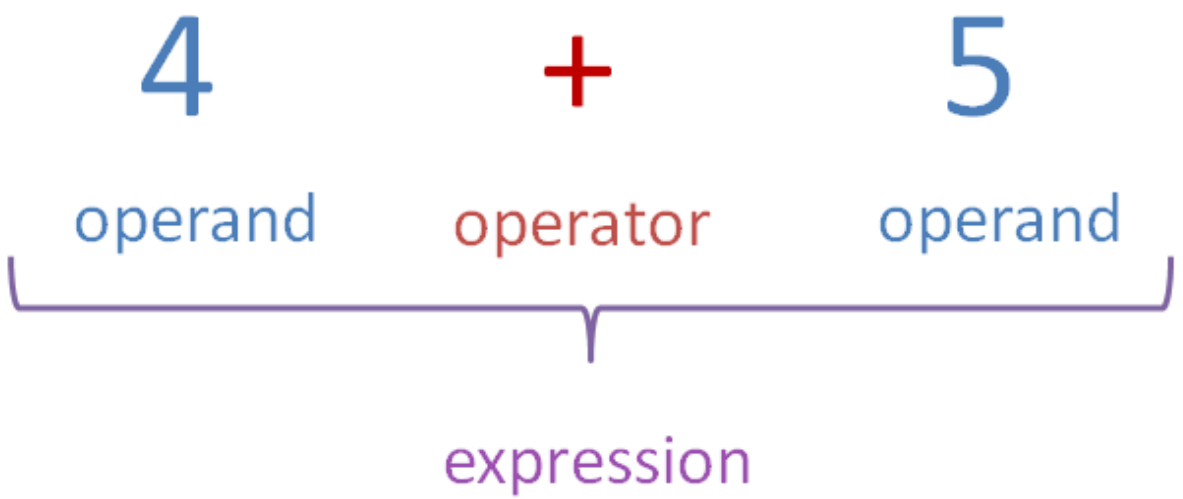
- **f-strings advantages:** More readable and efficient

```
# Can include expressions inside {}
score = 85
print(f"Your grade is {score}% - {'Pass' if score >= 60 else 'Fail'}")
```

💡 *Best Practice: Use f-strings for cleaner, more readable code (Python 3.6+)*

Operators & Expressions

We have seen how to store data (variables), now we see how to manipulate data



Classified by:

- Type of the operands
- Type of the result
- Number of operands

We will focus on the three most frequently used groups of operators:

- Arithmetic operators
- Logical operators
- Comparison operators

Operators & Expressions

Arithmetic Expressions

```
a = 2.0
b = 3.0

a + b     # addition
a * b     # multiplication
b ** a    # power
b % a     # modulo
b += a    # increment and assign
b *= a    # multiply and assign
```

```
5.0
6.0
9.0
1.0
b = 5.0
b = 6.0    # 10.0 if you ran the previous
```

Operators & Expressions

Logical (Boolean) Expressions

```
a = True
b = False

a and b     # conjunction
a or b      # disjunction
not a       # negation
not b
```

```
False
True
False
True
```

Operators & Expressions

Comparisons

```
a = 4
b = 5

a < b        # less than
a >= b       # greater or equal
a == b       # is equal to
a != b       # is different from
```

```
True
False
False
True
```

Operators & Expressions

Expression evaluation rules

```
a = 1.0
b = 2.0

c = a*a + b*b
```

```
c = 5.0
```

⟶ multiplication has precedence over addition

```
d = a * (a + b) * b
```

```
c = 6.0
```

⟶ parenthesis can change the order of evaluation

Operators & Expressions

Evaluation rules

```
i = 0
j = 10


# Comparison precedes logical operations (try)
a = i < j or j >= 5
b = (i < j) or (j >= 5)


# Arithmetic operators precede comparison (try)
c = i + 5 < j / 3.
d = (i + 5) < (j / 3.)


# AND precedes OR
e = a or b
f = a or b and c
g = (a or b) and c
```

```
a = True
b = True
c = False
d = False
e = True
f = True
g = False
```

Operators & Expressions

Expression evaluation rules

- Operators can be combined to form expressions, e.g.:

```
value > 5**2 + 4*3
```

- Order of evaluation of the expressions:
  - Arithmetic operators
  - Comparison operators
  - Logical (or Boolean) operators
  - Within each group, from left to right
  - Parenthesis can change the order of evaluation

Operators & Expressions

Arithmetic expressions with strings

```
a = 'str'
b = 'ing'
a + b
```

```
'string'
```

⟶ + concatenates strings

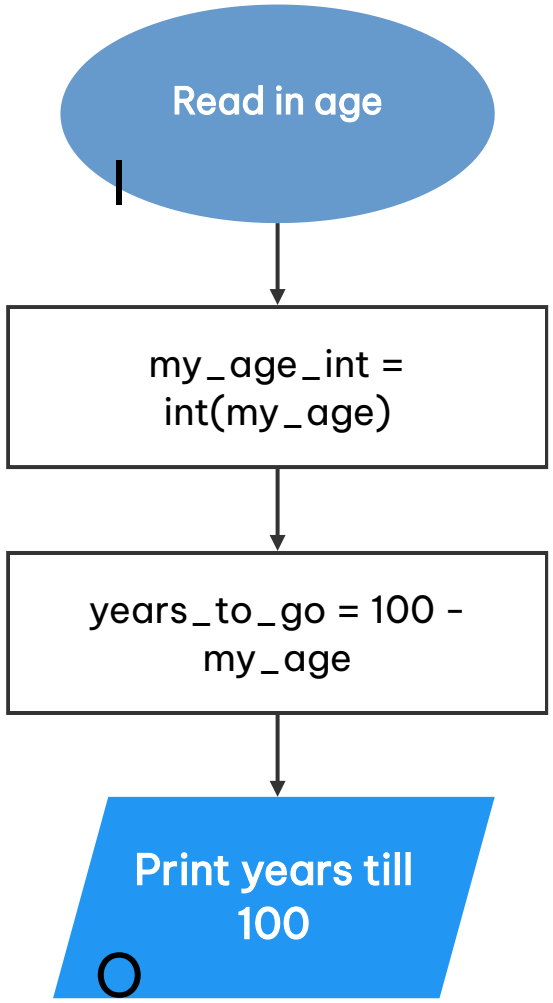
```
a * 5
```

```
'strstrstrstrstr'
```

⟶ * concatenates a string with itself n times

Control Structures

Flow Charts




```
my_age  = input('How old are you?  ')
```

```
my_age_int = int(my_age)
```

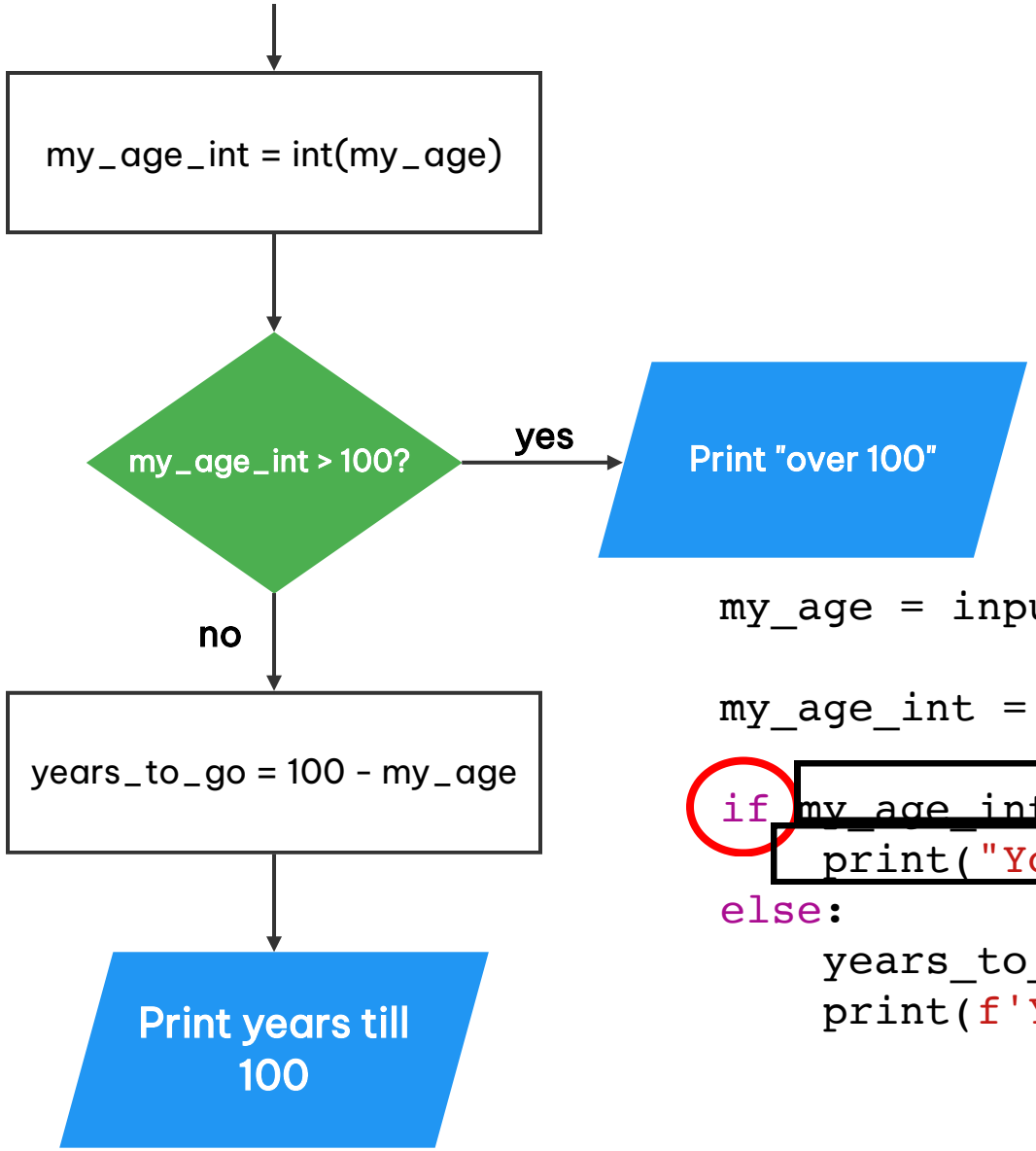
```
years_to_go  = 100 - my_age_int
```

```
print(f'You will turn 100 in {years_to_go} years.')
```

What if the person is 113 years old?

Control Structures

Branching / conditions (if / else)




```
my_age = input("What is your age?")

my_age_int = int(my_age)

if my_age_int > 100:
    print("You are already over 100 years old")
else:
    years_to_go = 100 - my_age_int
    print(f'You will turn 100 in {years_to_go} years.')
```

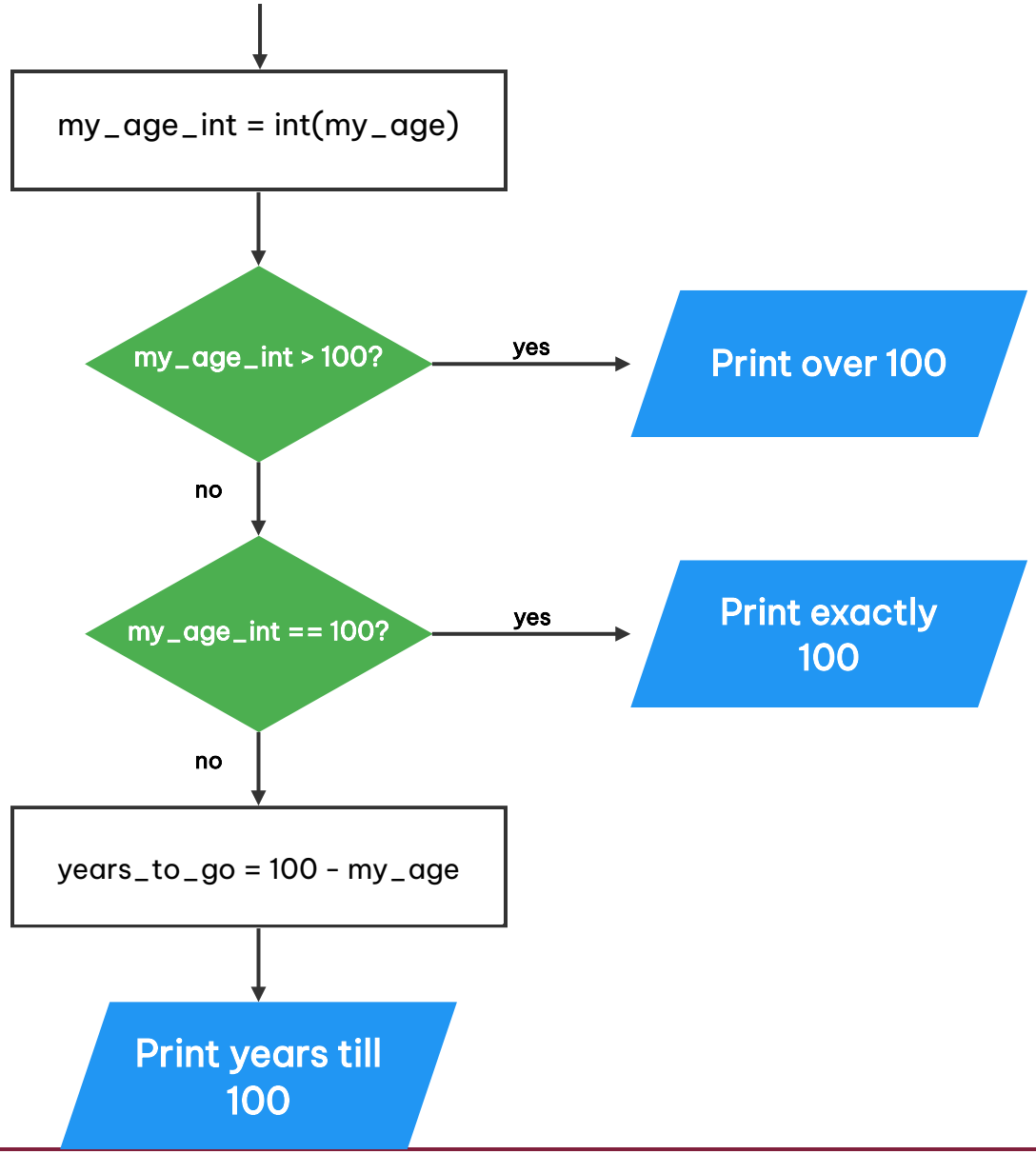
Condition

Statement

What if the person is exactly 100?

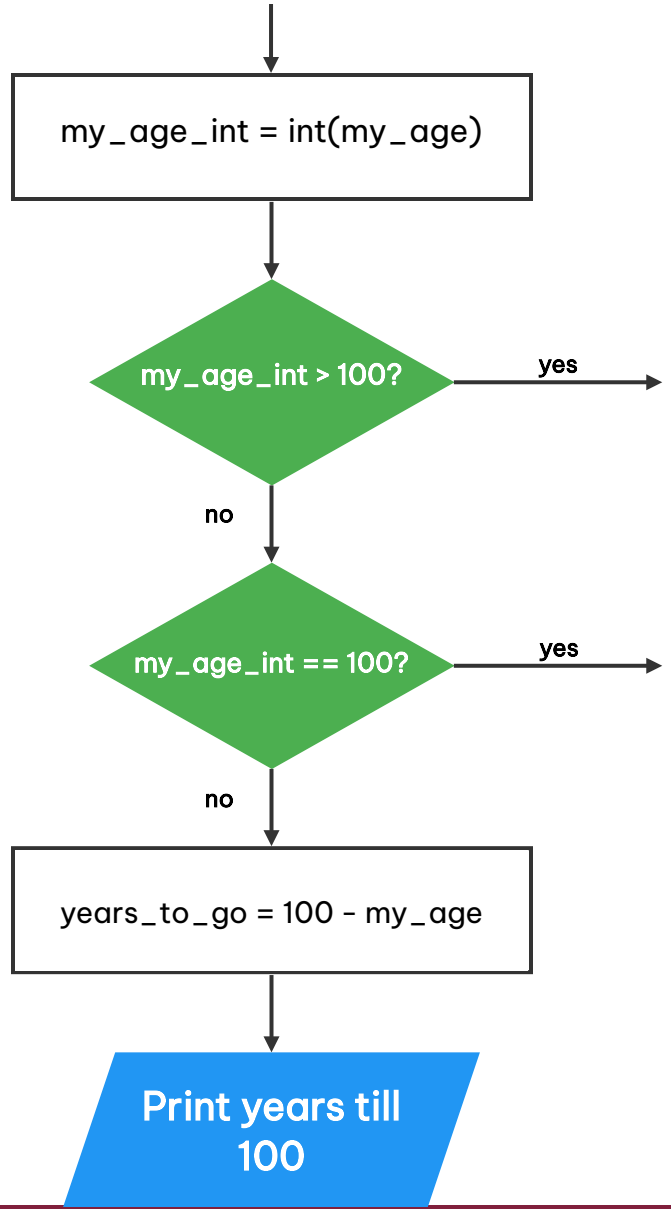
Control Structures

Branching / conditions (if / elif / else)



Control Structures

Branching / conditions (if / elif / else)




```
my_age = input("What is your age?")

my_age_int = int(my_age)

if my_age_int > 100:
    print("You are already over 100 years old")
elif my_age_int == 100:
    print("You are 100!")
else:
    years_to_go = 100 - my_age_int
    print(f'You will turn 100 in {years_to_go} years.')
```

Condition

Statement

Control Structures

Python Indentation

```
if condition:
    statement1
    statement2 ...

elif condition:
    statement4
    statement5 ...

else:
    statement6
    statement7 ...
```

Blocks of code

Indentation (4 spaces as per PEP-8)

Correct indentation is **mandatory**, not **optional** as in some other languages! This is for your own good (just try to read someone else's code).

Common Error:

```
if condition:
print("This will cause IndentationError!")
```

Correct:

```
if condition:
    print("Properly indented!")
```

- **Indentation defines code blocks** - no curly braces { }
- **Use 4 spaces** per indentation level (PEP-8 standard)
- **Be consistent** - mixing tabs and spaces causes errors
- **Colon (:)** at the end of control statements

Control Structures

for loops

- Iterate through the contents of a sequential data structure.

Example:

```
demo_list = ['life', 42, 'the Universe', 6, 'and', 7, 'everything']

for item in demo_list:
    print(f'The current item is: {item}')
```

```
The current item is: life
The current item is: 42
The current item is: The Universe
The current item is: 6
The current item is: and
The current item is: 7
The current item is: Everything
```

Control Structures

for loops

- Iterate through the contents of a sequential data structure.

Example:

```
demo_list = [2, 4, 6, 8]
n = 0

for num in demo_list:
    n += num
    print(f'The sum is: {n}')
```

```
The sum is: 2
The sum is: 6
The sum is: 12
The sum is: 20
```

Control Structures

for loops

- Iterate through the contents of a sequential data structure.

```
for variable_name in list_of_values:
    statements
```

Control Structures

for loops

Compute the sum of squares over a given range of numbers.

e.g.: n=10

ssq = 1*1 + 2*2 + ... + 10*10

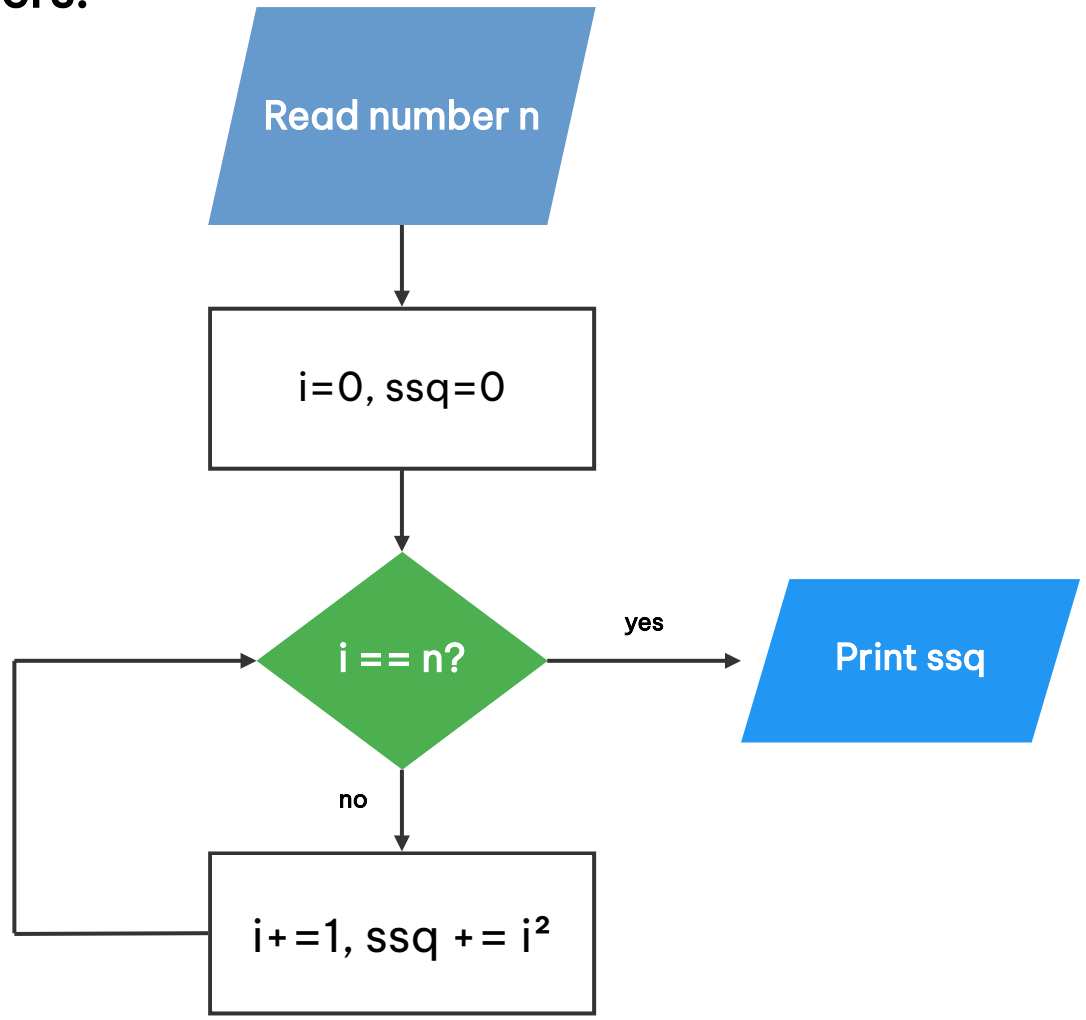
or:

ssq = 1**2 + 2**2 +... + 10**2

$$\text{ssq} = \sum_{i=1}^{n} i^2$$

```
n = int(input("Enter n: "))
ssq = 0
for i in range(1, n+1):
    ssq += i * i
print(f"Sum of squares: {ssq}")

# Alternative using sum() and list compre
ssq = sum(i**2 for i in range(1, n+1))
```



Control Structures

for loops

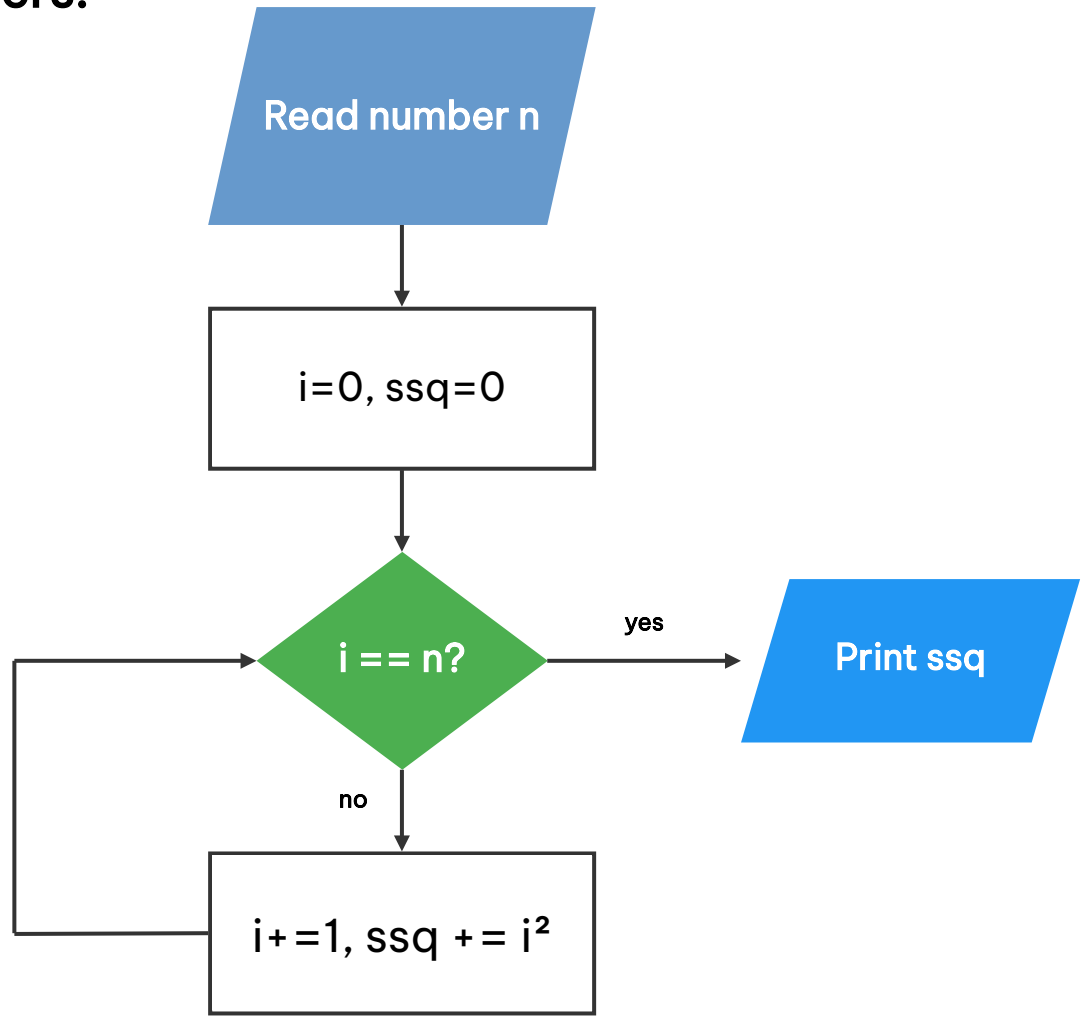
Compute the sum of squares over a given range of numbers.

```
n = input('Please enter a number: ')
n_int = int(n)
ssq = 0
                    start    stop     step
for i in range(1, n_int+1, 1):          12345678910
    ssq += i**2

print(f"The sum of squares up to {n_int} is {ssq}")
```

```
n = int(input("Enter n: "))
ssq = 0
for i in range(1, n+1):
    ssq += i * i
print(f"Sum of squares: {ssq}")

# Alternative using sum() and list comprehension
ssq = sum(i**2 for i in range(1, n+1))
```



Key Concepts:

- **Loop initialization:** Set counter and accumulator
- **Loop condition:** Continue while $i \leq n$
- **Loop body:** Add i^2 to running sum
- **Loop increment:** Increase counter by 1

Control Structures

Compute the sum of squares over a given range of numbers.

```
n = input('Please enter a number: ')
n_int = int(n)
ssq = 0
for i in range(1, n_int+1, 1):
    ssq += i**2
print(f"The sum of squares up to {n_int} is {ssq}")
```

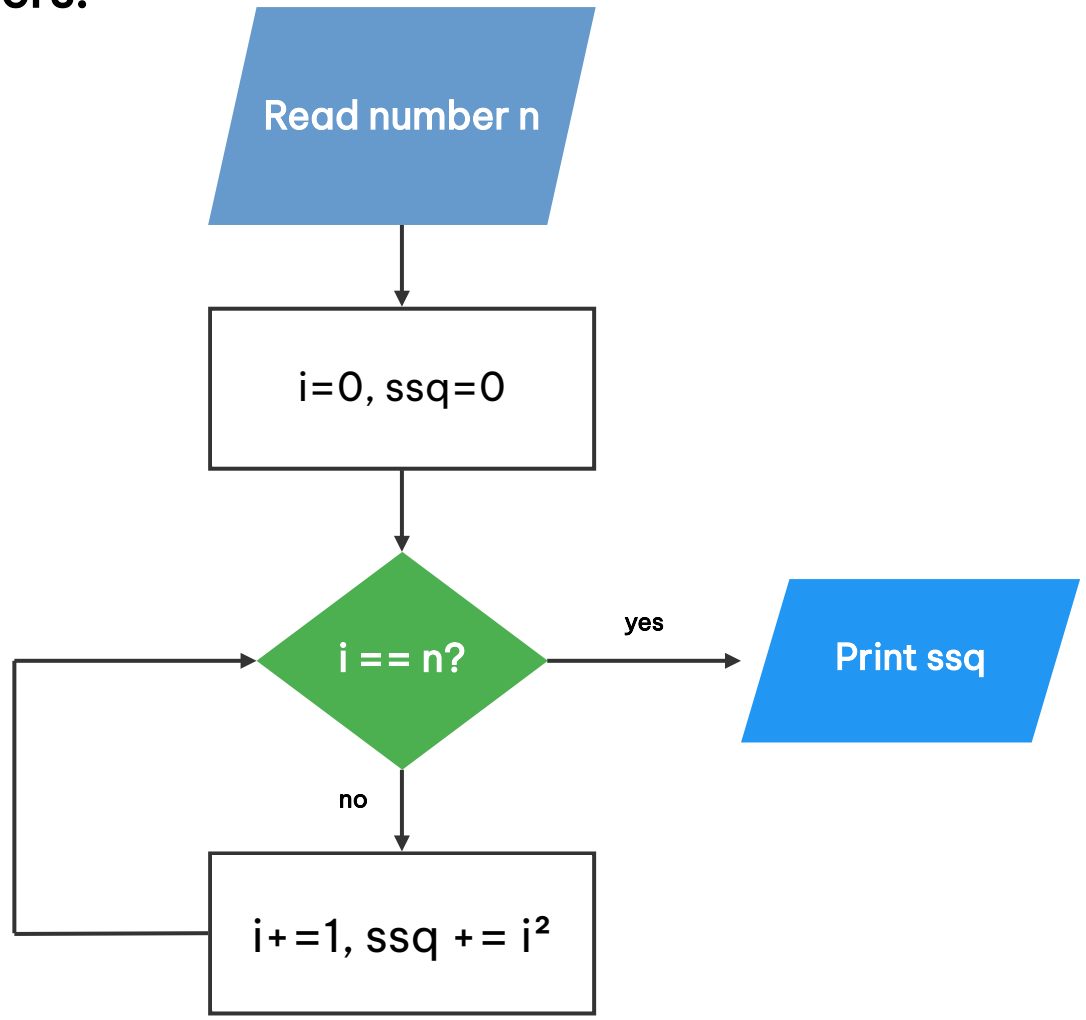
variable name

list of variables

statement

```
n = int(input("Enter n: "))
ssq = 0
for i in range(1, n+1):
    ssq += i * i
print(f"Sum of squares: {ssq}")

# Alternative using sum() and list comprehension
ssq = sum(i**2 for i in range(1, n+1))
```



Key Concepts:

- **Loop initialization:** Set counter and accumulator
- **Loop condition:** Continue while $i \leq n$
- **Loop body:** Add i^2 to running sum
- **Loop increment:** Increase counter by 1

Control Structures

for loops

```
n = input('Please enter a number: ')
n_int = int(n)
ssq = 0
for i in range(1, n_int+1, 1):
    ssq += i**2
print(f"The sum of squares up to {n_int} is {ssq}")
```

variable name

list of variables

statement

```
for variable_name in list_of_values:
    statements
```

```
n = int(input("Enter n: "))
ssq = 0
for i in range(1, n+1):
    ssq += i * i
print(f"Sum of squares: {ssq}")

# Alternative using sum() and list comprehension
ssq = sum(i**2 for i in range(1, n+1))
```

Key Concepts:

- **Loop initialization:** Set counter and accumulator
- **Loop condition:** Continue while $i \leq n$
- **Loop body:** Add i^2 to running sum
- **Loop increment:** Increase counter by 1

Control Structures

while loops

- Repeat an operation until a condition is met

Example: password protection

```
password = 'pass'

while password != 'python':
    print('Incorrect, try again')
    password = input('Enter password: ')

print('Correct!')
```

condition

statement

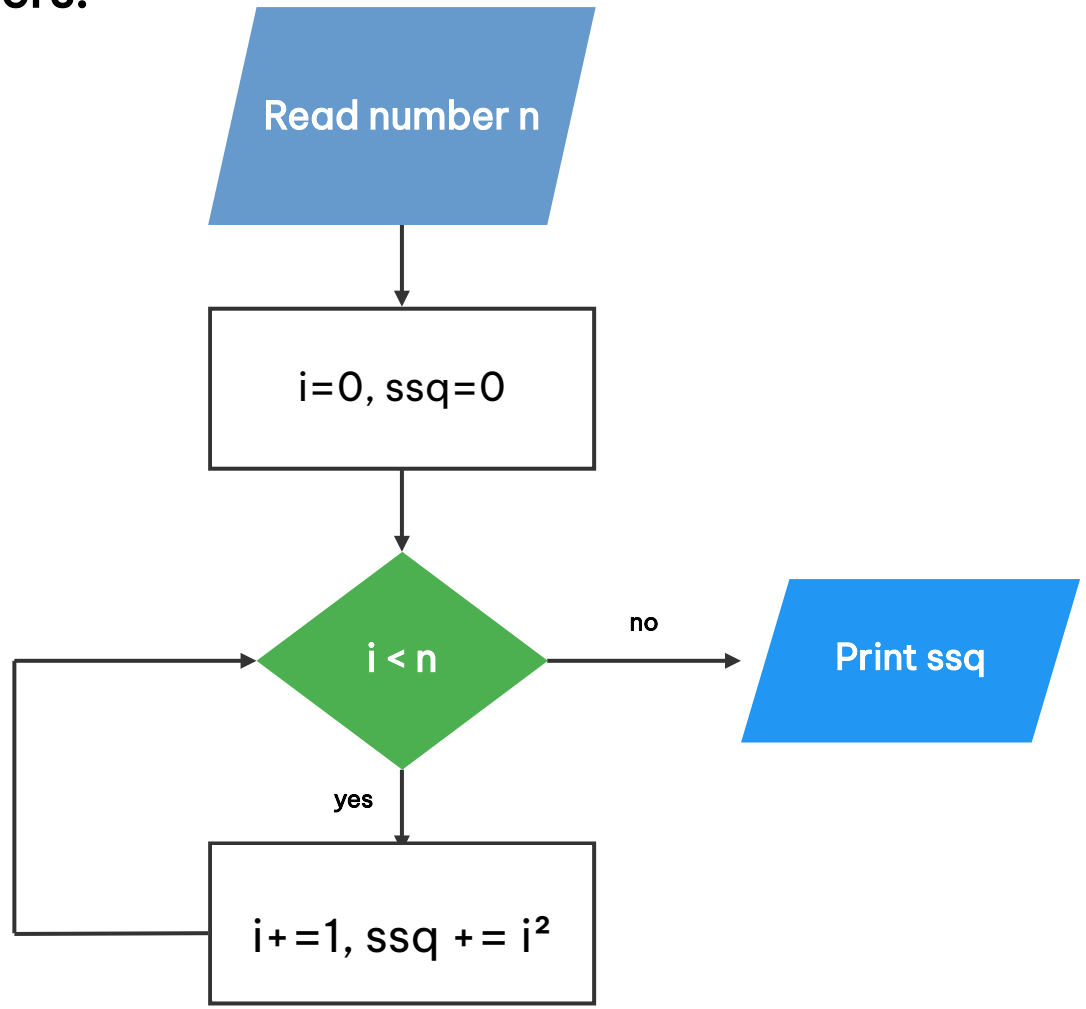
Control Structures

Compute the sum of squares over a given range of numbers.

```
n = input('Please enter a number: ')
n_int = int(n)
ssq = 0
i = 0

while i < n_int:
    ssq += i**2
    i += 1

print(f"The sum of squares up to {n_int} is {ssq}")
```



Key Concepts:

- **Loop initialization:** Set accumulator
- **Loop condition:** Continue while $i \leq n$
- **Loop body:** Add i^2 to running sum

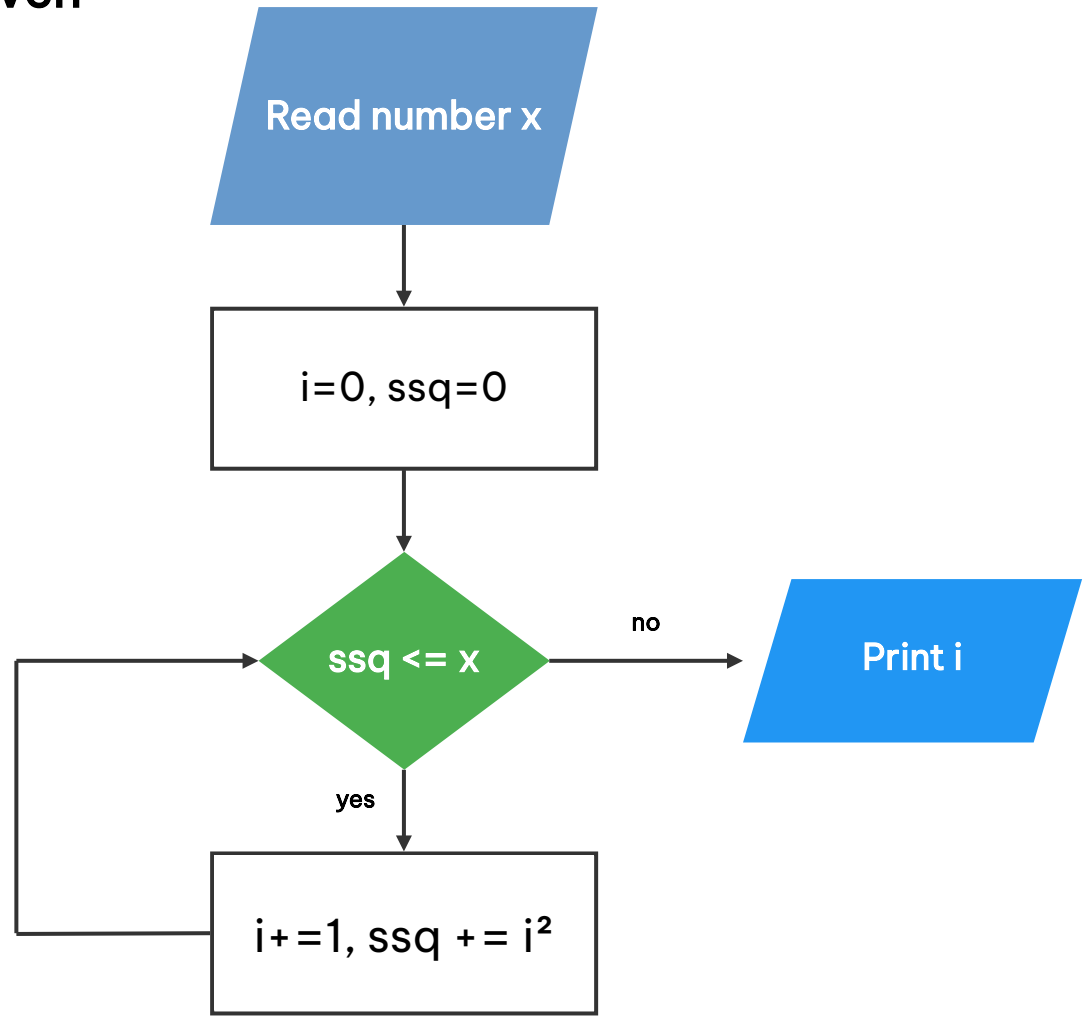
Control Structures

Compute the sum of squares until the total exceeds a given target value x.

```
x = int(input('Please enter a number: '))
ssq = 0
i = 0

while ssq <= x:
    ssq += i**2
    i += 1

print(f"The minimum n with ssq > {x} is {i}")
```



Key Concepts:

- **Loop initialization:** Set accumulator
- **Loop condition:** Continue while ssq $\leq$ x
- **Loop body:** Add i^2 to running sum

Control Structures

Compute the sum of squares until the total exceeds a given target value x.

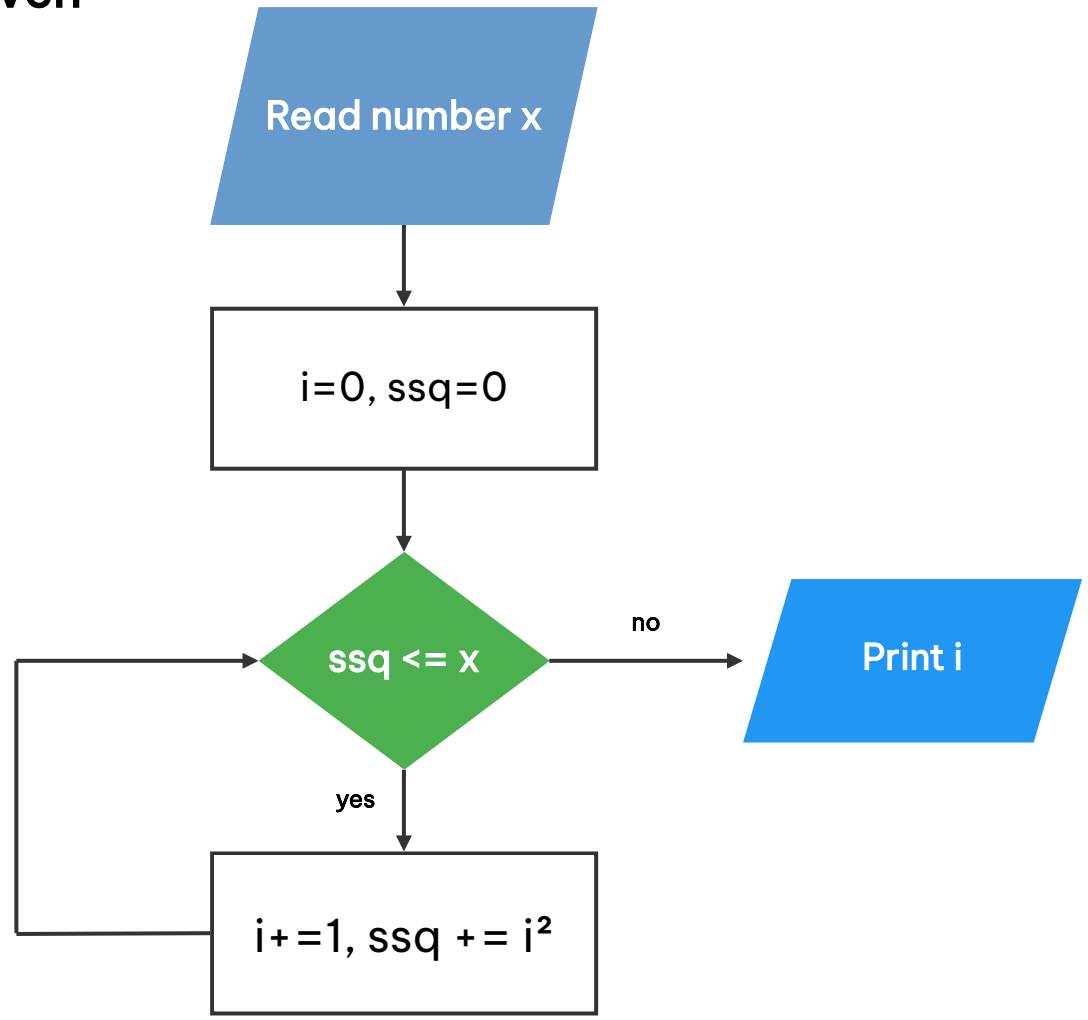
```
x = int(input('Please enter a number: '))
ssq = 0
i = 0

while ssq <= x:
    ssq += i**2
    i += 1

print(f"The minimum n with ssq > {x} is {i}")
```

condition

statement



Control Structures

⚠️ Beware of the infinite loops!!! ⚠️

```
while 1 == 1:
    print("Help! I'm stuck in a loop!")
```

```
x = 0.1
```

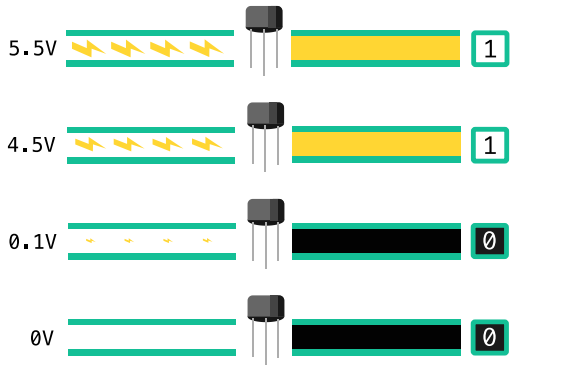
x should be integer

```
while x != 10:
    print(x)
    x += 1
```

Problem: Floating point arithmetic can cause precision issues. Starting with x=0.1 and adding 1 repeatedly will never exactly equal 10, causing an infinite loop!

Control Structures

Side note: *floating point precision*



Transistors store binary (0/1) states in hardware

Hardware Representatic

64-bit IEEE 754 standard:

- **1 bit**: Sign (+/-)
- **11 bits**: Exponent (scale)
- **52 bits**: Mantissa (precisic

```
# Classic precision problem
result = 0.1 + 0.2
print(result)
# Output: 0.30000000000000004

# Why? Binary cannot exactly represent 0.1
print(f"{0.1:.60f}")
# 0.1000000000000000055511151231...

# Comparison problems
0.1 + 0.2 == 0.3  # False!

# Solution: Use tolerance
import math
math.isclose(0.1 + 0.2, 0.3)  # True
```

Many decimals cannot be represented exactly in binary, like 1/3 = 0.333...

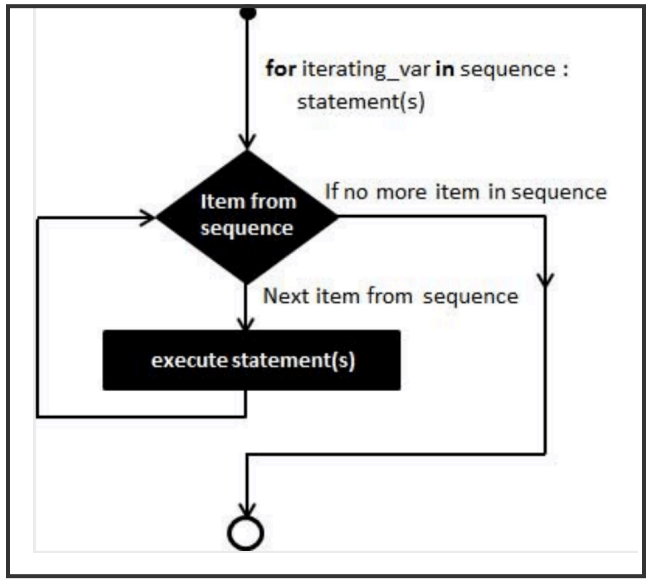
Common Problems in Scientific Computing

Key Takeaway: Never use == to compare floating point numbers. Always use tolerance-based comparisons (`math.isclose()`, `numpy.allclose()`) especially when analyzing experimental data where measurement precision matters.

Control Structures

Loops

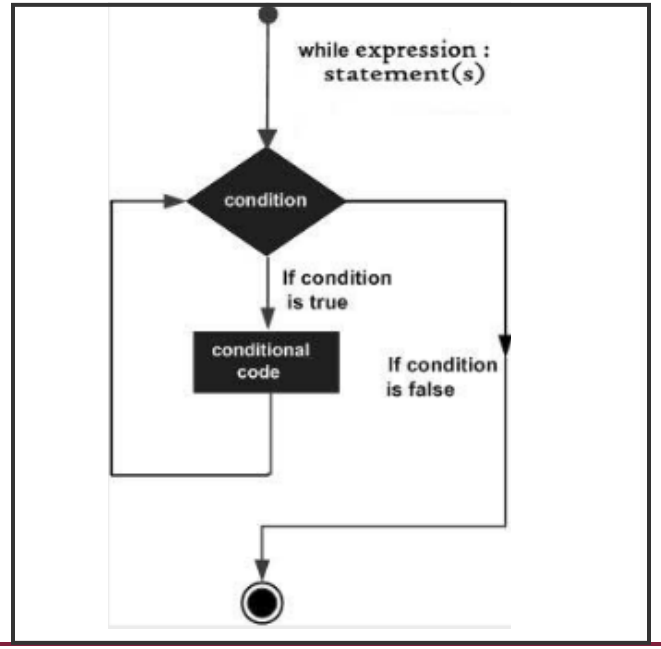
```
for variable in variable_list:
    statement 1
    statement 2
```



➡ iterates over some given values

```
# For loop example
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    print(num * 2)
```

```
while condition:
    statement 1
    statement 2
```



➡ iterates as long as the condition is true

```
# While loop example
count = 0
while count < 5:
    print(count)
    count += 1
```

Key Differences:

- **for:** Use when you know the sequence/range
- **while:** Use when condition-based iteration is needed

Control Structures

Break and **Continue**

```
x = 0.1

while x != 10:
    if type(x) != int:
        break
    print(x)
    x += 1
```

Check data type and exit the loop if inconsistent

Control Structures

Break and **Continue**

```
word = input("Please enter a word: ")
letter = input("Enter the letter you want to skip: ")

print("Your new word is: ")
for l in word:
    if l == letter:
        continue
    print(l)
```

Continues the loop (next step)

```
Please enter a word: python
Enter the letter you want to skip: h
Your new word is:
p
y
t
o
n
```

Control Structures

Break and **Continue**

```
word = input("Please enter a word: ")
letter = input("Enter the letter you want to skip: ")

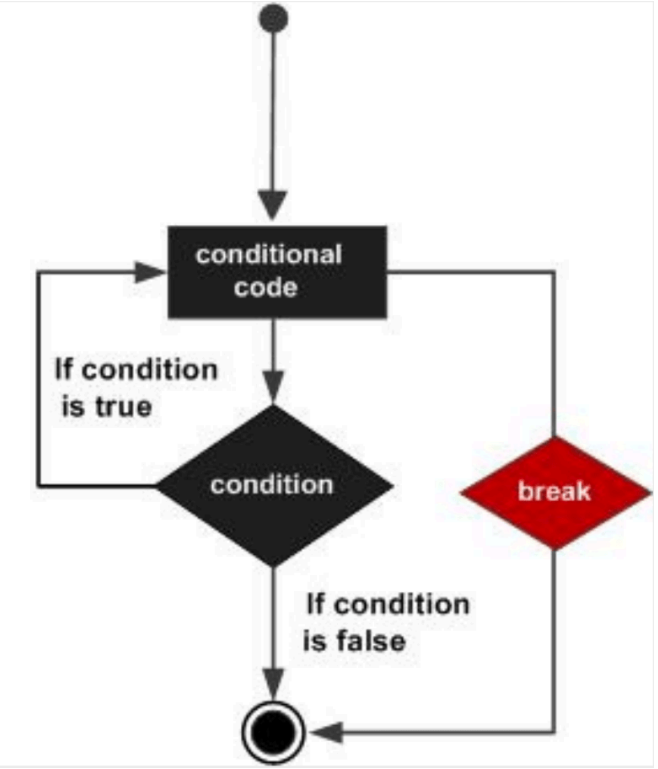
print("Your new word is: ")
for l in word:
    if l == letter:
        break
    print(l)
```

Continues the loop (next step)

```
Please enter a word: python
Enter the letter you want to skip: h
Your new word is:
p
y
t
```

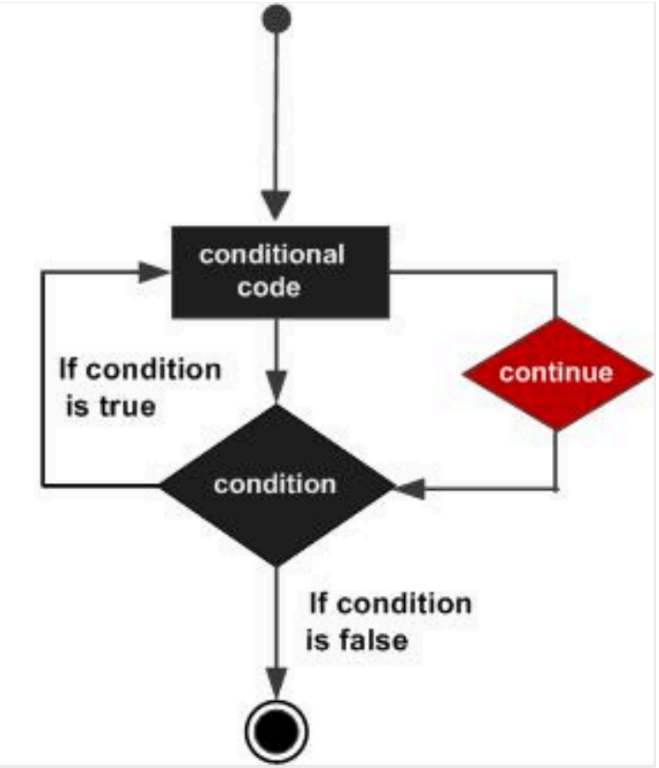
Control Structures

Loops: *Flow control*



Breaks out of the loop, without executing the next code blocks

(exit the loop prematurely)



Continues the loop, without executing remaining code blocks within the loop

(begin the next iteration without waiting for the end of the current loop)

Control Structures

Clever Looping

- **enumerate** – Getting Index and Element

The `enumerate()` function extracts both the index and element from any iterable.

❌ The Hard Way:

```
fruits = ['apple', 'banana', 'cherry']

for i in range(len(fruits)):
    print(f"{i}: {fruits[i]}")
```

✅ The Pythonic Way:

```
fruits = ['apple', 'banana', 'cherry']

for i, fruit in enumerate(fruits):
```

```
0: apple
1: banana
2: cherry
```

💡 **Tip:** You can start counting from any number: `enumerate(fruits, start=1)`

Control Structures

Clever Looping

- **enumerate** – Getting Index and Element
- **zip** – Combining multiple iterables

The `zip()` function returns tuples containing the i-th element from each iterable.

```
names = ['Alice', 'Bob', 'Charlie']
ages = [25, 30, 35]
cities = ['Lisbon', 'Porto', 'Coimbra']

# Iterate over multiple lists simultaneously
for name, age, city in zip(names, ages, cities):
    print(f"{name} is {age} years old and lives in {city}")

# Create a list of tuples
pairs = list(zip(names, ages))
```

```
Alice is 25 years old and lives in Lisbon
Bob is 30 years old and lives in Porto
Charlie is 35 years old and lives in Coimbra
[('Alice', 25), ('Bob', 30), ('Charlie', 35)]
```

⚠ **Note:** `zip()` stops when the shortest iterable is exhausted.

Control Structures

Clever Looping

- **enumerate** – Getting Index and Element
- **zip** – Combining multiple iterables
- **dict iteration** – items, keys, values

In Python 3, use `.items()`, `.keys()`, and `.values()` methods (not the old `iter*` versions).

✅ dict.items() - Key-Value Pairs

```
student_grades = {
    'Alice': 85,
    'Bob': 92,
    'Charlie': 78
}

# Iterate over key: value pairs
for name, grade in student_grades.items():
    print(f"{name}: {grade}")

# Iterate over keys only
for name in student_grades.keys():
    print(f"Student: {name}")

# Iterate over values only
```

```
Alice: 85
Bob: 92
Charlie: 78

Student: Alice
Student: Bob
Student: Charlie

Grade: 85
Grade: 92
Grade: 78
```

Control Structures

Clever Looping

🧬 Example 1: Processing Biological Data

```
genes = ['BRCA1', 'TP53', 'EGFR']
expressions = [2.5, 1.8, 3.2]
cell_types = ['breast', 'lung', 'brain']
```

```
Sample 1: BRCA1 expression is 2.5 in breast cells
Sample 2: TP53 expression is 1.8 in lung cells
Sample 3: EGFR expression is 3.2 in brain cells
```

📊 Example 2: Data Analysis with Dictionary

```
experiment_results = {
    'control': [1.2, 1.5, 1.1],
    'treatment_A': [2.1, 2.3, 1.9],
    'treatment_B': [1.8, 2.0, 1.7]
}

# Calculate averages for each condition
```

```
control: average = 1.27
treatment_A: average = 2.10
treatment_B: average = 1.83
```

🎯 **Key Takeaway:** These techniques make code more compact, readable and efficient

Control Structures

Clever Looping

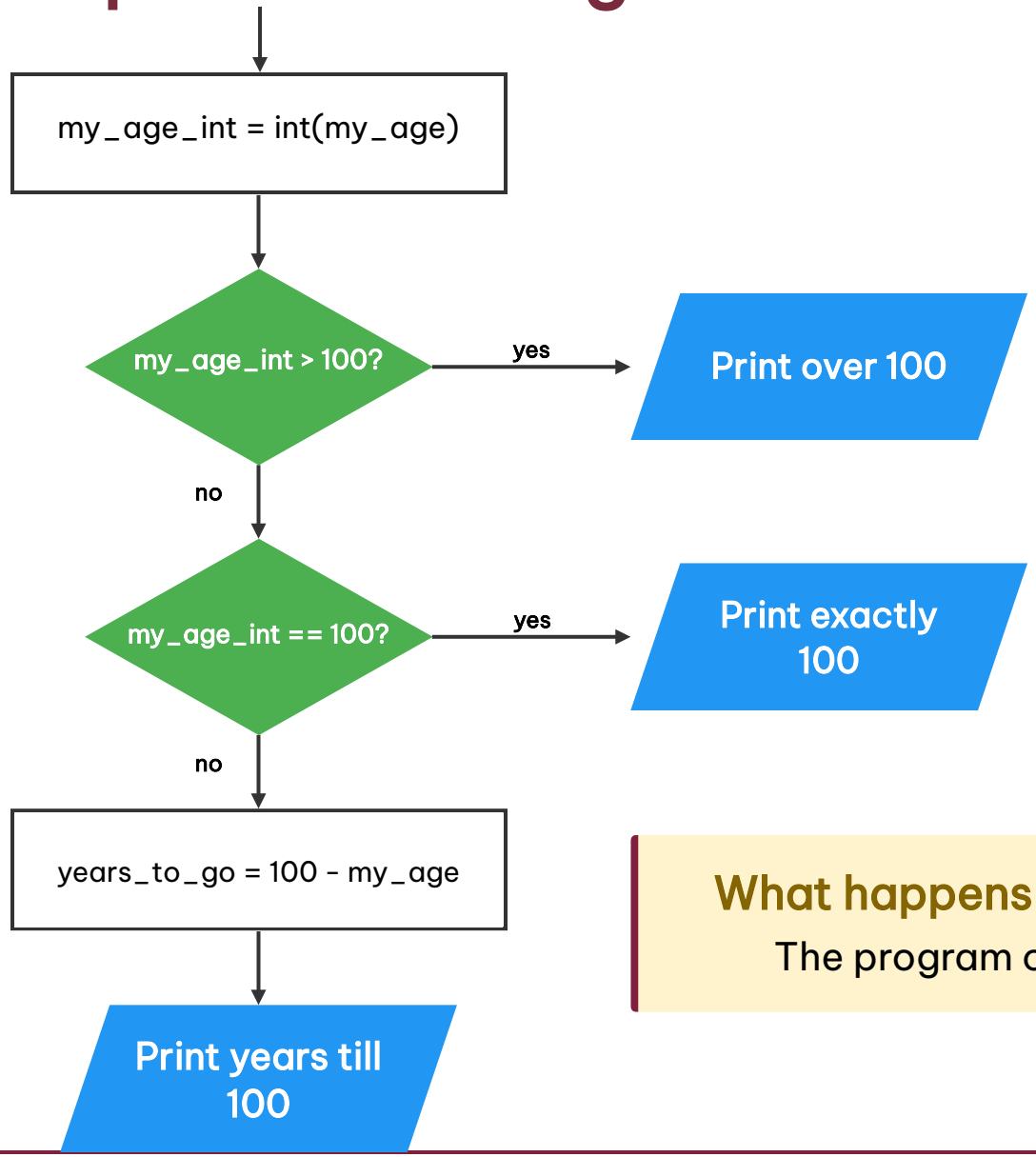
📋 Summary:

- **enumerate()**
  Get index + element together
- **zip()**
  Combine multiple iterables
- **dict.items()**
  Iterate key-value pairs
- **dict.keys()**
  Iterate dictionary keys
- **dict.values()**
  Iterate dictionary values

✨ Benefits:

- **More readable** code
- **Less error-prone** than manual indexing
- **Pythonic style** - follows best practices
- **Better performance** in many cases
- **Easier to debug** and maintain

Exception Handling



Giving the user a second chance

Without Exception Handling

The program crashes when invalid input is provided

```
my_age = input("Enter your age: ")
my_age_int = int(my_age)  # Crashes if user enters "abc"!

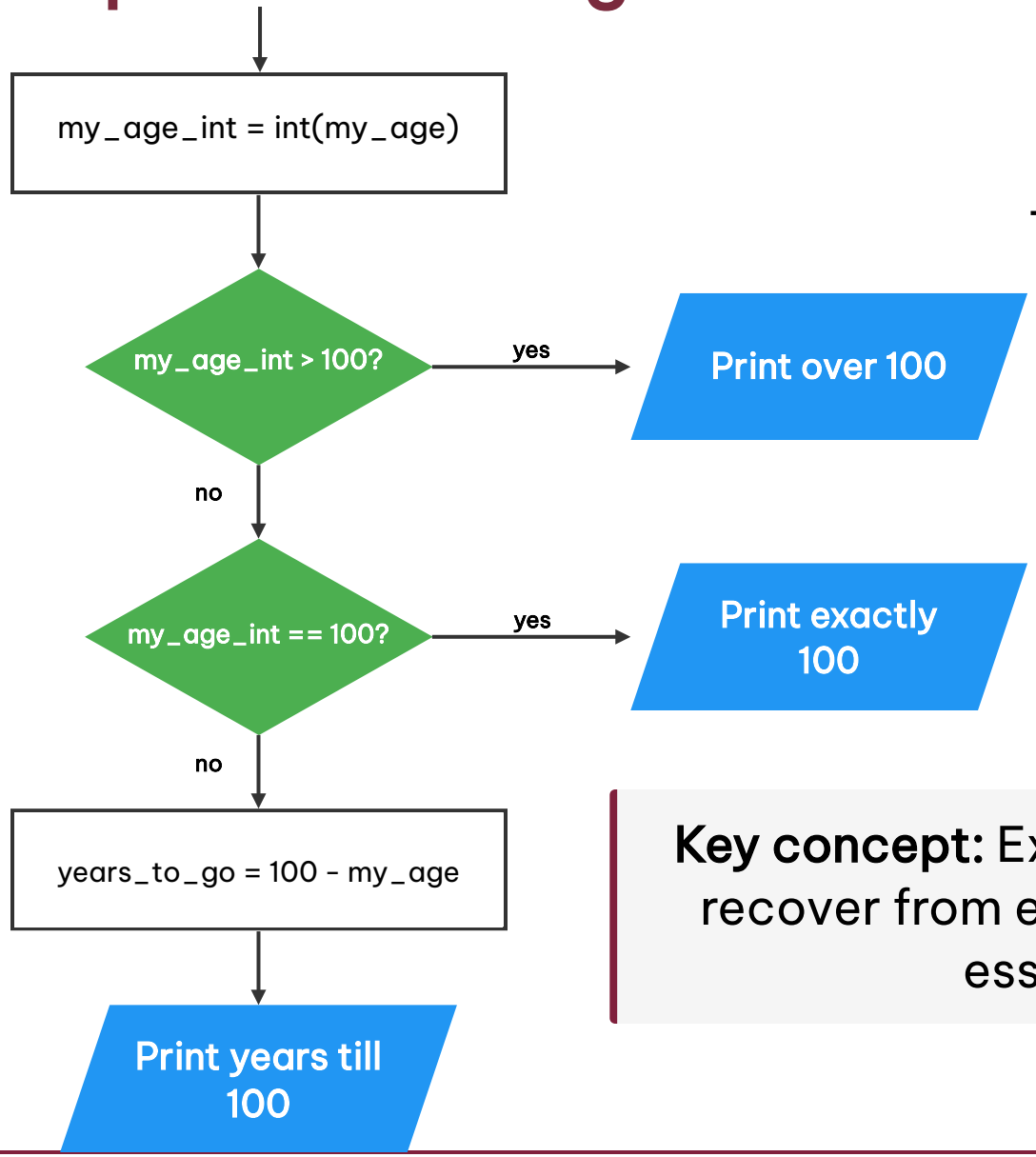
if my_age_int > 100:
    print("Over 100")
# ... rest of code never executes
```

What happens if the user enters a letter?

The program crashes with a `ValueError`!

Exception Handling

Giving the user a second chance



With Exception Handling

The program handles errors gracefully and continues

```
my_age = input("Enter your age: ")

try:
    my_age_int = int(my_age)
    if my_age_int > 100:
        print("Over 100")
    # ... rest of logic
except ValueError:
    print("Error: Please enter a valid number")
    # Program continues gracefully
```

Key concept: Exception handling allows programs to anticipate and recover from errors, making code more robust and user-friendly - essential for processing experimental data.

Exception Handling

Syntax and common exception types

Try-Except Syntax

```
try:
    # Code block that might cause an error
    risky_operation()

except ExceptionType:
    # What should happen when the exception occurs
    handle_error()
```

- Specify exception type to catch specific errors
- Program continues running after exception is handled

Common Exception Types

`ValueError`

Invalid conversion: `int("abc")`

`ZeroDivisionError`

Division by zero: `10 / 0`

`NameError`

Variable not defined: `print(undefined_var)`

`IOError`

File operations fail: `open("nonexistent.txt")`

Practical Example: Handling Multiple Exceptions

```
try:
    data = open("experiment_data.csv")
    sample_size = int(input("Enter sample size: "))
    mean_value = total / sample_size
except IOError:
    print("Error: Data file not found")
except ValueError:
    print("Error: Invalid number entered")
except ZeroDivisionError:
    print("Error: Sample size cannot be zero")
```

Functions and Modular Programming

What are functions?

❌ The Problem:

Often programs contain sequences of statements that are carried out multiple times

```
Statement 1
Statement 2
Statement 3
Statement 4
Statement 5
Statement 2
Statement 3
Statement 6
```

This makes the program: long, hard to follow, and prone to errors

✅ The Solution:

We can collect the repeated statements into **functions**

```
def MyFunction1:
    Statement 2
    Statement 3
```

```
Statement 1
MyFunction 1
Statement 4
Statement 5
MyFunction 1
Statement 6
```

🎯 Functions make code reusable, readable, and reliable!

Functions and Modular Programming

Types of Functions

🔧 Built-in Functions

Part of the Python language

```
# Examples of built-in functions
length = len([1, 2, 3, 4])  # 4
numbers = range(5)          # 0,1,2,3,4
absolute = abs(-42)         # 42
maximum = max(3, 7, 1)      # 7

print()
```

👨‍💻 User-Defined Functions

Created with the **`def`** keyword

```
# Example of user-defined function
def calculate_bmi(weight, height):
    """Calculate Body Mass Index"""
    bmi = weight / (height ** 2)
    return bmi

# Usage
my_bmi = calculate_bmi(70, 1.75)
```

Functions and Modular Programming

📚 Module Functions

Functions created by others and gathered into modules or libraries - access by importing

```
# Examples of module functions
import math
import statistics

# Math module functions
sqrt_value = math.sqrt(16)          # 4.0
sin_value = math.sin(math.pi/2)     # 1.0

# Statistics module functions
data = [1, 2, 3, 4]
mean_value = statistics.mean(data)  # 2.5
```

Functions and Modular Programming

Modules & Libraries

📚 What are **Modules** and **Libraries**?

Modules: Python files containing functions, classes, and variables that you can import and use

Libraries/Packages: Collections of modules organized together for specific purposes

📥 Import Methods

```
# 1. Import entire module
import math
result = math.sqrt(16)  # 4.0

# 2. Import with alias
import numpy as np
array = np.array([1, 2, 3])

# 3. Import specific functions
from math import sqrt, pi
result = sqrt(16)  # 4.0
area = pi * 2**2   # 12.566

# 4. Import all (avoid this!)
from math import *  # Not recommended
```

🔧 Creating Custom Modules

```
# File: bio_utils.py
def gc_content(dna_sequence):
    """Calculate GC content of DNA"""
    gc = dna_sequence.count('G') + dna_sequence.count('C')
    return (gc / len(dna_sequence)) * 100

def reverse_complement(dna_sequence):
    """Get reverse complement"""
    complement = {'A':'T', 'T':'A', 'C':'G', 'G':'C'}
    return ''.join(complement[base]
                   for base in reversed(dna_sequence))

# Usage in another file:
import bio_utils
gc = bio_utils.gc_content("ATGCGC")  # 66.67
```

💡 **Best Practice:** Use descriptive aliases (`import numpy as np`) and import only what you need for cleaner, more readable code!

Functions and Modular Programming

Defining a Function

To define a function in Python, we must:

- **Declare it as a function** using `def`
- **Name it** meaningfully
- **Define what input it has** (its arguments or parameters)
- **Define what output it has** (if any)
- **Write the statements** that transform the input into the output
- **Provide documentation** to describe how to use the function (optional but recommended)

🌡️ Example: Temperature Conversion

```
1 def convert_celsius_to_fahrenheit(temp_celsius):
2     """Converts a temperature in Celsius to Fahrenheit""
3     temp_fahrenheit = (9.0/5.0) * temp_celsius + 32.0
4     return temp_fahrenheit
```

Functions and Modular Programming

Defining a Function

To define a function in Python, we must:

- **Declare it as a function** using `def`
- **Name it** meaningfully
- **Define what input it has** (its arguments or parameters)
- **Define what output it has** (if any)
- **Write the statements** that transform the input into the output
- **Provide documentation** to describe how to use the function (optional but recommended)

📋 Function Anatomy:

```
def function_name(parameter1, parameter2):
    """Optional docstring to describe the function"""
    # Function body - statements that do the work
    return result  # Optional return statement
```

Functions and Modular Programming

The anatomy of a Function

📋 Function Anatomy:

```
def function_name(parameter1, parameter2):
    """Optional docstring to describe the function"""
    # Function body - statements that do the work
    return result  # Optional return statement
```

Function Header

Syntax: `def function_name(arguments):`

- All functions begin with `def`
- Function name follows Python naming rules
- Arguments in parentheses, separated by commas
- Colon (:) ends the header

Function Naming Conventions

- **Meaningful:** describe what it does
- **Imperative mood:** verb describing action
- **Unambiguous:** clear purpose
- **Simple:** if hard to name, break it up

Good examples:

```
calculate_area()
convert_temperature()
analyze_spike_train()
```

Functions and Modular Programming

The anatomy of a Function

📋 Function Anatomy:

```
def function_name(parameter1, parameter2):
    """Optional docstring to describe the function"""
    # Function body - statements that do the work
    return result  # Optional return statement
```

Function Header

Syntax: `def function_name(arguments):`

- All functions begin with `def`
- Function name follows Python naming rules
- Arguments in parentheses, separated by commas
- Colon (:) ends the header

Documentation (Docstring)

Optional but highly recommended

```
def convert_celsius_to_fahrenheit(temp_celsius):
    """Converts temperature in Celsius to Fahrenheit

    Args:
        temp_celsius: Temperature in Celsius

    Returns:
        Temperature in Fahrenheit
    """
```

Used by `help()` function and development tools

Function Naming Conventions

- **Meaningful:** describe what it does
- **Imperative mood:** verb describing action
- **Unambiguous:** clear purpose
- **Simple:** if hard to name, break it up

Good examples:

```
calculate_area()
convert_temperature()
analyze_spike_train()
```

Functions and Modular Programming

The anatomy of a Function

📋 Function Anatomy:

```
def function_name(parameter1, parameter2):
    """Optional docstring to describe the function"""
    # Function body - statements that do the work
    return result  # Optional return statement
```

Function Header

Syntax: `def function_name(arguments):`

- All functions begin with `def`
- Function name follows Python naming rules
- Arguments in parentheses, separated by commas
- Colon (:) ends the header

Return Statement

```
# Multiple return values
def add_and_multiply(x, y):
    """Adds and multiplies two numbers"""
    return x + y, x * y

a = 3.0
b = 4.0
sum_result, product_result = add_and_multiply(a, b)
print(f"Sum: {sum_result}, Product: {product_result}")  # Sum: 7.0, Product: 12.0
```

Functions and Modular Programming

Using Functions

- **Within Same Script:** Define the function first, then call it anywhere below in your code

```
def convert_celsius_to_fahrenheit(temp_celsius):
    """Converts a temperature in Celsius to Fahrenheit"""
    return (9.0/5.0) * temp_celsius + 32.0

# Using the function
user_temp = input("Please enter a temperature in Celsius: ")
user_temp = float(user_temp)
temp_f = convert_celsius_to_fahrenheit(user_temp)
print(f"The temperature in Fahrenheit is: {temp_f}")
```

Important: Variable names don't need to match between function definition and function call. The function parameter `temp_celsius` receives the value from `user_temp`.

- **From Other Scripts (Modules):** Save functions in separate files and import them for reuse across projects

```
# Save function in conversions.py file, then import it
import conversions as cnv

user_temp = input("Please enter a temperature in Celsius: ")
user_temp = float(user_temp)
temp_f = cnv.convert_celsius_to_fahrenheit(user_temp)
print(f"The temperature in Fahrenheit is: {temp_f}")

# Alternative: Import specific function directly
from conversions import convert_celsius_to_fahrenheit
temp_f = convert_celsius_to_fahrenheit(user_temp)  # Use without prefix
```

Functions and Modular Programming

Flexible inputs

Function arguments can be fixed or flexible (can even be functions)

- **Default Values:** Provide typical values for optional parameters
- **Keyword Arguments**: Specify parameter names to change order
- **Variable length arguments (*args)**: For functions that need to process an unknown number of inputs - converted into a tuple which is empty if *args are not specified

Important: Required arguments must come before default arguments

Example: matplotlib's `hist()` function has 15+ parameters with defaults - this flexibility makes scientific libraries powerful and user-friendly

Functions and Modular Programming

Variables in a program may not be accessible at all locations - it depends on **where they were declared**

The ***Scope*** of a variable in Python can be:

- **Global:** Defined outside functions, accessible everywhere
- **Local:** Defined inside functions, only accessible within that function

Local Scope

```
def calculate_area(radius):
    # These are LOCAL variables
    pi = 3.14159
    area = pi * radius ** 2
    return area

# This would cause an ERROR:
# print(pi)    # NameError: pi not defined
# print(area)  # NameError: area not defined

result = calculate_area(5)
print(f"Area: {result}")  # This works!
```

Global Scope

```
# This is a GLOBAL variable
laboratory_name = "CNC-UC Coimbra"

def print_lab_info(researcher):
    # Can READ global variables
    print(f"{researcher} works at {laboratory_name}")

    # But cannot MODIFY them directly
    # laboratory_name = "New"  # Creates LOCAL copy!

print_lab_info("Ana")  # Works!
print(laboratory_name)  # Also works!
```

Functions and Modular Programming

Variables in a program may not be accessible at all locations - it depends on **where they were declared**

The ***Scope*** of a variable in Python can be:

- **Global:** Defined outside functions, accessible everywhere
- **Local:** Defined inside functions, only accessible within that function

Best Practices for Variable Scope

- **Keep functions self-contained:** Pass all needed data as parameters
- **Use return statements:** Send results back rather than modifying global variables
- **Avoid global keyword:** Makes code harder to debug and test
- **Functions should not have side-effects:** They should only transform input to output

Functions and Modular Programming

Summary: Functions

- Functions allow us to ***gather code*** that we will use in more than one place so we do not have to repeat it
- This makes our programs **quicker to write**, **easier to read** and less likely to have **errors**
- They help **break down complex problems** and make our code **reusable** in multiple projects
- Functions can have **multiple inputs** and **multiple outputs**
- Anything can be an input or return value of a function
- Default and keyword arguments allow flexible function usage

Importing Modules

- Modules are Python files containing functions and variables
- Use `import module_name` to access external code
- Alias imports: `import numpy as np`
- Specific imports: `from math import sqrt`
- Python's standard library provides extensive functionality

Key Takeaway: essential programming concepts for building modular, reusable code. Functions are the building blocks of well-organized programs, allowing you to write cleaner, more maintainable scientific code. Understanding scope, imports, variables, operations and control structures provides the foundation for doing some real work

Thank you!





Course materials (Part I) adapted from:

- Moritz Helias
- Abigail Morrison
- Yury Zaitsev
- Renato Duarte



